

Chapter 6. Streams

Node streams have a reputation for being hard to understand and work with. That might have been the case in the early days of Node, but things have changed. Today, it's relatively easy to create and consume streams in Node. We can even use native JavaScript async iterators and generators to work with Node streams.

In this chapter, I'll explain the concept of streams, why they're needed, and how to create, use, and combine them to efficiently process large amounts of data without overwhelming the memory available to a Node process.

Introducing Streams

When you download a file from the internet, watch a show, or listen to a song, you are using streams. The content is being streamed to you one chunk at a time.

Streams are basically collections of data, similar to arrays or strings, but instead of storing data in memory space, streams process data over time. You can use streams to process very large amounts of data using a limited memory space.

Analogies for streams are all around us in life. When you have a sink full of dirty dishes, you can load them up in a dishwasher, and that would be processing all of them at once, like holding them in an array. If you can't use a dishwasher, you'll need to employ the concept of streams and clean up the sink one dish at a time. You take one dish, scrub it, rinse it, and then move on to the next one.

This is great if you have a small number of dishes to wash. If you have a lot of dishes to wash and someone who can help, you can use two streams! You can have one person scrub and the other rinse. Each person deals with a different task, but the input of one process (the rinsing) depends on the output of another (the scrubbing). We can add more streams from there, such as drying and stacking. That's how multiple streams can be combined to make tasks more efficient.

When we use the output of one stream as the input for another, that's known as *pipng*. That's the same term used in the Unix OS as *pipes* are used to transfer data from one process to another.

Here's an example for using pipes in Linux. Suppose you are tasked with counting the occurrence of the word `require` in all files under a big project. In Linux, you have the `grep` command that can find patterns in files and the `wc` command that can count lines, words, or characters. We can use the output of the `grep` command and pipe it as the input of the `wc` command:

```
$ grep -wR require * | wc -l
```

Pipes in Linux enable us to compose powerful commands from smaller commands, and the ability to pipe streams on each other gives them the power of composability as well.

If we had a stream to `grep` and a stream to `wc` in Node, we can combine them like this:

```
const grep = ... // A stream for the grep output
const wc = ... // A stream for the wc input

grep.pipe(wc)
```

Or, to put the dishwashing analogy in pseudocode:

```
const scrub = ... // An output stream of scrubbed dishes
const rinse = ... // An input stream for dishes to be rinsed
```

```
scrub.pipe(rinse)
```

We'll talk more about the `pipe` method later in the chapter.

Before we conclude the dishwashing analogy, let's use it to understand a couple more concepts about streams.

There are two major types of streams in Node: readable streams and writable ones. Some streams are both readable and writable; these are known as duplex streams.

A readable stream produces output, and a writable stream needs input. In this analogy, the scrub process is the readable part (it has an output of scrubbed dishes), and the rinse process is the writable part (it needs an input of scrubbed dishes). When using the `pipe` method in Node, the caller has to be a readable stream, and the argument has to be a writable stream:

```
readableStream.pipe(writableStream)
```

Data flow in streams is another concept you should understand. As long as the two people washing the dishes are handling their tasks at a relatively similar rate, the flow of dishes from dirty to scrubbed to rinsed works fine. But let's say the scrubber needs to work on one particularly dirty dish and scrub it really well; the rinser in that case might need to pause a bit before they get the next dish. Similarly, if the rinser is behind, a pile of scrubbed dishes might start to accumulate, and the scrubber might need to pause. These are some of the challenging aspects of using streams. We'll talk more about that shortly.

Using Streams

To really understand the powerful efficiency of streams, let's see some numbers. Let's go over one practical example of doing the same task with and without streams, and compare memory usage.

First, we need a big file. Streams are all about processing large amounts of data. Here's a simple way to create a big file in Node:

```
import fs from 'node:fs';
import { randomBytes } from 'node:crypto';

const file = fs.createWriteStream('./big.file');

for (let i = 0; i <= 1e6; i++) {
  file.write(
    randomBytes(200).toString('hex')
  );
}

file.end();
```

Note what I used to create that big file: a writable stream! The `node:fs` module can be used to read from and write to files using a stream interface. We created this big file one chunk at a time, using a loop. In every iteration, we *streamed* one line to the file, and we repeated that 1 million times.

When I ran this code, it generated a 381 MB file.

Now, let's say that we need to make this file available for download from a Node web server. Here's an example web server designed to exclusively serve *big.file*:

```
import { createServer } from 'node:http';
import { readFile } from 'node:fs/promises';

const server = createServer();

server.on('request', async (req, res) => {
  const data = await readFile('./big.file');

  res.end(data);
});

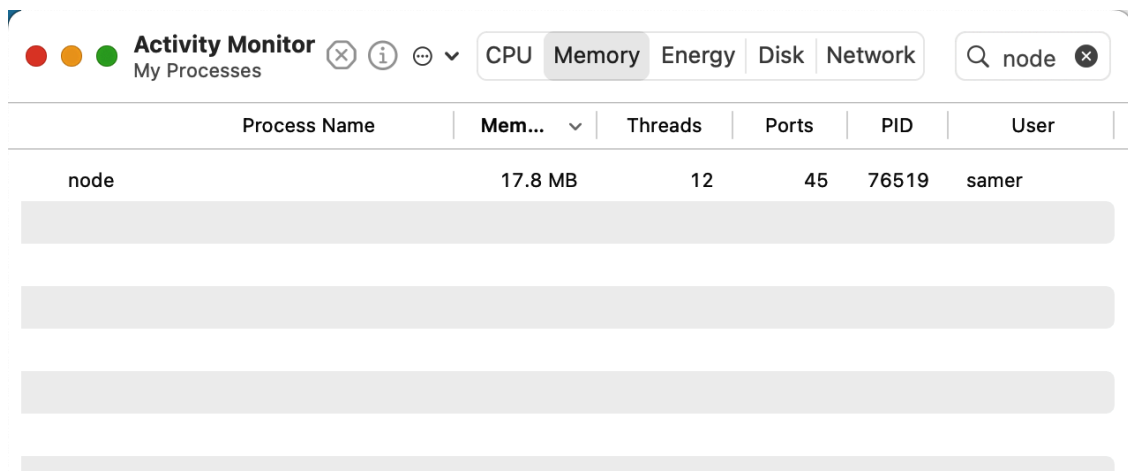
server.listen(3000, () => {
```

```
console.log('Server is running...');
});
```

When this web server gets a request, it'll serve the big file content using the `fs.readFile` method. I didn't need to use streams to make that happen, and the code uses asynchronous methods, so we're not blocking the call stack. So what exactly is the problem?

Well, let's see what happens when we run the server, connect to it, and monitor the memory while doing so.

When I ran the server, it started out with a normal amount of memory for a process—17.8 MB, as shown in [Figure 6-1](#).

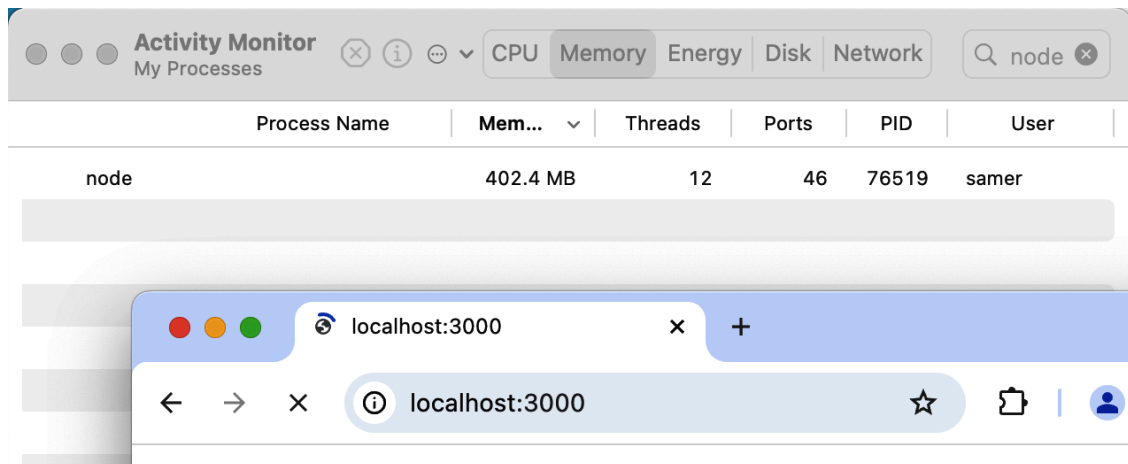


The screenshot shows the macOS Activity Monitor application with the 'Memory' tab selected. A search bar at the top right contains the text 'node'. The table below lists the 'node' process with the following details:

Process Name	Mem...	Threads	Ports	PID	User
node	17.8 MB	12	45	76519	samer

Figure 6-1. The server process's initial memory

Then I connected to the server. [Figure 6-2](#) shows what happened to the memory consumed—it jumped to 402.4 MB.



The screenshot shows the macOS Activity Monitor application with the 'Memory' tab selected. A search bar at the top right contains the text 'node'. The table below lists the 'node' process with the following details:

Process Name	Mem...	Threads	Ports	PID	User
node	402.4 MB	12	46	76519	samer

In the foreground, a web browser window is open with the address bar showing 'localhost:3000'.

Figure 6-2. The server process's memory after a request

We basically put the whole *big.file* content in memory before we wrote it out to the response object. That is very inefficient, and we can do much better by using streams.

Luckily, many of Node's built-in modules use streams under the hood, and the `node:http` module is one of them. The `res` object in the `request` event handler is a writable stream! We can use it to stream data to the client instead of sending it all at once. We just need a readable stream to represent the content of *big.file*, and we can then use the `pipe` method to send the file one chunk at a time without consuming around 400 MB of memory.

The `node:fs` module can give us a readable stream for any file using the `createReadStream` method. Here's how we can use it with the `res` stream object:

```
import { createServer } from 'node:http';
import { createReadStream } from 'node:fs';

const server = createServer();

server.on('request', (req, res) => {
  const src = createReadStream('./big.file');
  src.pipe(res);
});

server.listen(3000, () => {
  console.log('Server is running...');
});
```

To test that, after you run the server, connect to it to download the file, and watch the memory consumption of the process while you do (see [Figure 6-3](#)).

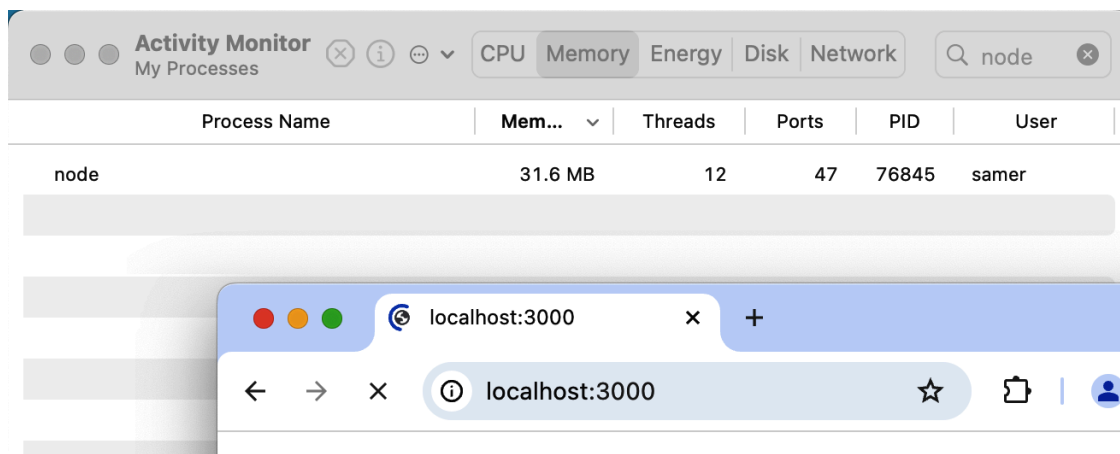


Figure 6-3. The server process's memory with streams

With this version of the web server, when a client asks for that big file, we stream it one chunk at a time, which means we don't buffer it in memory at all.

You can push this example to its limits. Regenerate *big.file* with 5 million lines instead of just 1 million, which would take the file to well over 2 GB. In most Node installations, that would be even bigger than the default buffer limit.

If you try to serve that bigger file using `fs.readFile`, you simply can't by default (you can change the limits). But with `fs.createReadStream`, there is no problem at all streaming 2 GB of data to the requester and, best of all, the process memory usage will be roughly the same.

Fundamentals of Streams

As we learned in the previous section, streams can be readable, writable, or both. A readable stream is an abstraction for a source from which data can be consumed. An example of that is the `fs.createReadStream` method. A writable stream is an abstraction for a destination to which data can be written. An example of that is the `fs.createWriteStream` method.

A stream that is both readable and writable is known as a *duplex* or *bidirectional* stream. These streams are useful where data needs to flow in both directions, such as in network communication using TCP sockets.

A transform stream is a duplex stream that can be used to modify the data in transit. An example of that is the `zlib.createGzip` stream that compresses data using gzip. You can think of a transform stream as a function where the input is the writable stream part and the output is the readable stream part.

Many of the built-in modules in Node implement the streaming interface.

[Table 6-1](#) lists some examples.

Table 6-1. Examples of stream objects

Readable streams	Writable streams
HTTP requests	HTTP responses
<code>fs</code> read streams	<code>fs</code> write streams
<code>zlib</code> streams	<code>zlib</code> streams
<code>crypto</code> streams	<code>crypto</code> streams
TCP sockets	TCP sockets
child-process <code>stdout</code> & <code>stderr</code>	child-process <code>stdin</code>
<code>process.stdin</code>	<code>process.stdout</code> , <code>process.stderr</code>

Notice that the stream objects in [Table 6-1](#) are closely related. Also note how the `stdio` streams (`stdin`, `stdout`, `stderr`) have the inverse stream types when it comes to child processes. This allows for a really easy way to pipe to, and from, these child-process `stdio` streams using the main-process `stdio` streams. We'll learn about child processes in [Chapter 7](#).

All streams are instances of `EventEmitter`. They emit events that can be used to read and write data. However, we can consume streams data in a simpler way using the `pipe` method.

The pipeline Method

The `pipe` method in Node connects two stream objects: a readable source and a writable destination. This is the important line that you need to remember:

```
readableSrc.pipe(writableDest);
```

In this simple line, we're piping the output of a readable stream (the source of data) as the input of a writable stream (the destination). The source has to be a readable stream, and the destination has to be a writable one. Of course, they can both be duplex/transform streams as well. In fact, if we're piping into a duplex stream, we can chain pipe calls just like we do in Linux:

```
readableSrc
  .pipe(transformStream1)
  .pipe(transformStream2)
  .pipe(finalWritableDest);
```

The `pipe` method returns the destination stream. That allows for chaining if the destination stream is readable as well.

For streams `a` (readable), `b` and `c` (duplex), and `d` (writable), we can do the following:

```
a.pipe(b)
  .pipe(c)
  .pipe(d);

// Which is equivalent to:
a.pipe(b);
b.pipe(c);
c.pipe(d);

// Which, in Linux, is equivalent to:
// $ a | b | c | d
```

The `pipe` method is the easiest way to consume streams as it automatically manages the flow of data so that a writable destination is not overwhelmed by a readable source. When we use other stream methods to read/write data, we'll need to manually manage the flow of data.

Usually, when we're using the `pipe` method, we don't need to use stream events other than the `error` events, but if we need to consume the streams in more custom ways, we can combine other stream methods and events to do that.

An even better way to consume multiple streams is to use the `pipeline` method. You can use it with both the callback and promise patterns:

```
import { pipeline } from 'node:streams/promises';

await pipeline(
  readableSrc,
  transformStream1,
  transformStream2,
  finalWritableDest
);
```

Besides the more clear syntax, the `pipeline` method is better at handling stream errors and doing any needed cleanups.

TIP

You should use the `pipeline` function within a `try/catch` statement to handle any errors thrown by any streams in the pipeline.

Stream Events

We can use stream methods like `read` and `write` in combination with stream event listeners to consume streams.

Here's the simplified equivalent code of what the `pipe` method mainly does to read and write data:

```
// readable.pipe(writable)

readable.on('data', chunk => {
  writable.write(chunk);
});

readable.on('end', () => {
  writable.end();
});
```

[Table 6-2](#) shows a list of the important events and methods that can be used with readable and writable streams.

Table 6-2. Important events and methods for streams

	Readable streams	Writable streams
Events	data, end, error, close, readable	drain, finish, error, close, pipe, unpipe
Methods	pipe(), unpipe(), wrap(), destroy()	write(), destroy(), end()
	read(), unshift(), resume(), pause(), setEncoding()	cork(), uncork(), setDefaultEncoding()

The events and methods in [Table 6-2](#) are somehow related because they are usually used together.

Events and methods can be combined for custom use of streams. To consume a readable stream, we can use the `pipe / unpipe` methods or the `read / unshift / resume` methods. To consume a writable stream, we can make it the destination of `pipe / unpipe`, or just write to it with the `write` method and call the `end` method when we're done.

When not using `pipe / unpipe`, the use of stream events will be needed. The most important events on a readable stream are as follows:

The `data` event

Emitted whenever the stream passes a chunk of data to the consumer

The `end` event

Emitted when there is no more data to be consumed from the stream

The most important events on a writable stream are as follows:

The `finish` event

Emitted when all data has been flushed to the underlying system

The `drain` event

A signal that the writable stream can receive more data

The `drain` event is important. It's needed to ensure a balanced flow of data. The rate at which data is pushed by or written to a stream is known as the *pressure* of the stream. When a writable stream has a slower rate of processing data than the rate of a readable stream that's pushing data, data will start buffering in the writable stream. This condition is known as *backpressure*.

As long as a writable stream is operating within its memory limits, its `write` method will return `true`. When the memory limit is reached, the `write` method returns `false` to indicate that further attempts to write data to the stream should stop until the `drain` event is emitted.

The `error` event may be emitted by streams at any time and should always be handled, even when using the `pipe` method:

```
readable.pipe(writable);

readable.on('error', (err) => {
  // Handle potential read errors
});

writable.on('error', (err) => {
```

```
// Handle potential write errors
});
```

The callback functions for handling `error` events receive a single `Error` object as an argument. This object can be used to handle different errors differently.

Paused and Flowing Modes

Readable streams have two main modes that affect the way we can consume them. They can be either in the *paused* mode or in the *flowing* mode. These modes are sometimes referred to as *pull* and *push* modes.

All readable streams start in the paused mode by default, but they can easily be switched to flowing and back to paused when needed. Sometimes, the switching happens automatically.

When a readable stream is in the paused mode, we can use the `read()` method to read from the stream on demand. However, for a readable stream in the flowing mode, the data is continuously flowing, and we have to listen to events to consume it.

In the flowing mode, data can actually be lost if no consumers are available to handle it. This is why when we have a readable stream in flowing mode, we need a `data` event handler. In fact, just adding a `data` event handler switches a paused stream into flowing mode, and removing the `data` event handler switches the stream back to paused mode. Some of this is done for backward compatibility with the older Node streams interface.

To manually switch between these two stream modes, we can use the `resume()` and `pause()` methods.

TIP

When consuming readable streams using the `pipe` or `pipeline` methods, we don't have to worry about these modes because they are managed automatically.

Implementing Streams

When we talk about streams in Node, there are two main tasks:

- *Implementing* the streams
- *Consuming* the streams

So far, we've been talking only about consuming streams. Let's implement some!

Modules that implement streams are usually the ones that import the `stream` module.

Writable Streams

To implement a writable stream, we use the `Writable` constructor from the `stream` module:

```
import { Writable } from 'node:stream';
```

We can implement a writable stream in many ways. For example, we can extend the `Writable` class:

```
class myWritableStream extends Writable {  
  // ...  
}
```

However, I prefer the simpler constructor approach. We just create an object from the `Writable` constructor and pass it a number of options. The only required option is a `write` function, which exposes the `chunk` of data to be written:

```
import { Writable } from 'node:stream';

const outputStream = new Writable({
  write(chunk, encoding, callback) {
    console.log(chunk.toString());
    callback();
  }
});

process.stdin.pipe(outputStream);
```

This `write` method takes three arguments:

The `chunk`

This is the streamed data. It's usually a Node buffer object but can also be a string.

The `encoding` argument

This is needed if `chunk` is a string to be encoded with an encoding different from the default `utf8`. You can omit it otherwise.

The `callback`

This is a function that we need to call after we're done processing the data chunk. It's what signals whether the write step was successful or not. To signal a failure, we call the callback function with an error object as its argument.

In this example, we simply `console.log` the `chunk` (as a string) and call the callback function after that without an error (to indicate success). This is a very simple echo stream. It simply echoes back anything it receives.

To consume this stream, we used it with `process.stdin`, which is a readable stream, so we can just pipe `process.stdin` into our `outputStream`.

When we run this code, anything we type into `process.stdin` will be echoed back using the `outputStream console.log` line.

The `outStream` object is actually similar to the built-in `process.stdout` object. We can simply pipe `stdin` into `stdout` and we'll get the exact same echo feature with this single line:

```
process.stdin.pipe(process.stdout);
```

Readable Streams

To implement a readable stream, we use the `Readable` constructor from the `stream` module:

```
import { Readable } from 'node:stream';

const inStream = new Readable();
```

There is a simple way to implement readable streams. We can just directly `push` the data that we want the consumers to consume:

```
// ...

inStream.push('ABCDEFGHIJKLM');
inStream.push('NOPQRSTUVWXYZ');

inStream.push(null); // No more data

inStream.pipe(process.stdout);
```

When we `push` a `null` object, that means we want to signal that the stream does not have any more data.

To consume this simple readable stream, we simply piped it into the writable `process.stdout` stream object.

When we run this code, we'll be reading all the data from `inStream` and echoing it to the standard output. Very simple, but also not very efficient. We're basically pushing all the data in the stream all at once and then piping it to `process.stdout`. The much better way is to push data on de-

mand, when a consumer asks for it. We can do that by implementing the `read()` method in a readable stream configuration:

```
const inStream = new Readable({
  read(size) {
    // There is a demand for the data...
    // Someone wants to read it.
  }
});
```

When the `read` method is called on a readable stream, the implementation can push partial data to the queue. For example, we can push one letter at a time, starting with character code `65` (which represents `A`), and increment the code on every push:

```
const inStream = new Readable({
  read(size) {
    this.push(String.fromCharCode(this.currentCharCode++));
    if (this.currentCharCode > 90) {
      this.push(null);
    }
  },
});

inStream.currentCharCode = 65;

inStream.pipe(process.stdout);
```

While the consumer is reading a readable stream, the `read` method will continue to fire, and we'll push more letters. We need to stop this cycle somewhere, and that's why I used an `if` statement to push `null` when the `currentCharCode` is greater than `90` (which represents `Z`).

This code is equivalent to the simpler one we started with, but now we're pushing data on demand when the consumer asks for it, as shown in

[Figure 6-4](#). You should always do that.

```
JS read-stream.js X
JS read-stream.js > ...
1  import { Readable } from 'node:stream';
2
3  const inStream = new Readable({
4    read(size) {
5      this.push(String.fromCharCode(this.currentCharCode++));
6      if (this.currentCharCode > 90) {
7        this.push(null);
8      }
9    },
10  });
11
12  inStream.currentCharCode = 65;
13
14  inStream.pipe(process.stdout);
15

TERMINAL bash + v [ ] [X] ...
● $ node read-stream.js
○ ABCDEFGHIJKLMNOPQRSTUVWXYZ $ |
```

Figure 6-4. A readable stream

If the source of data is a string, an array, or any other iterable object, you can use the `Readable.from` method to create a readable stream for it:

```
// To convert string data into a readable stream
Readable.from('ABCDEFGHIJKLMNOPQRSTUVWXYZ');
```

You can use JavaScript function generators with `Readable.from` and use the `yield` keyword to make more data readable with the created stream. Here's an example:

```
function* generate() {
  for (let index = 65; index <= 90; index++) {
    yield String.fromCharCode(index);
  }
}

// To create a stream from yielded values:
Readable.from(generate());
```

Duplex/Transform Streams

With `Duplex` streams, we can implement both readable and writable streams within the same object. It's as if we inherit from both interfaces.

Here's an example duplex stream that combines the preceding writable and readable examples:

```
import { Duplex } from 'node:stream';

const inoutStream = new Duplex({
  write(chunk, encoding, callback) {
    console.log(chunk.toString());
    callback();
  },

  read(size) {
    this.push(String.fromCharCode(this.currentCharCode++));
    if (this.currentCharCode > 90) {
      this.push(null);
    }
  }
});

inoutStream.currentCharCode = 65;

process.stdin.pipe(inoutStream).pipe(process.stdout);
```

By combining the methods, we can use this duplex stream to read the letters from `A` to `Z`, and we can also use it for its echo feature. We pipe the readable `stdin` stream into this duplex stream to use the echo feature, and we pipe the duplex stream itself into the writable `stdout` stream to see the letters `A` through `Z`.

It's important to understand that the readable and writable sides of a duplex stream operate completely independently from one another. This is merely a grouping of two features into one object.

A transform stream is a more interesting duplex stream because its output is computed from its input. For a transform stream, we don't have to implement the `read` or `write` methods; we only need to implement a `transform` method, which combines both of them. It has the signature of the `write` method, and we can use it to `push` data as well.

Here's a simple transform stream that echoes back anything you type into it after transforming it to uppercase format:

```
import { Transform } from 'node:stream';

const upperCaseTr = new Transform({
  transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase());
    callback();
  }
});

process.stdin.pipe(upperCaseTr).pipe(process.stdout);
```

In this transform stream, which we're consuming exactly like the previous duplex stream example, we implemented only a `transform()` method. In that method, we convert the `chunk` into its uppercase version and then `push` that version as the readable part, as shown in [Figure 6-5](#).

```
JS transform-stream.js X
JS transform-stream.js > ...
1  import { Transform } from 'node:stream';
2
3  const upperCaseTr = new Transform({
4    transform(chunk, encoding, callback) {
5      this.push(chunk.toString().toUpperCase());
6      callback();
7    },
8  });
9
10 process.stdin.pipe(upperCaseTr).pipe(process.stdout);
11
```

TERMINAL

node + - [] [X] ...

○ \$ node transform-stream.js

Hello World!

HELLO WORLD!

|

Figure 6-5. A transform stream

Async Generators and Iterators

We saw how a generator function can be used to create a readable stream. This works with async iterators as well, so you can use `for await` in an async generator function and use that function to create a readable stream.

Here's an example of a readable stream created from API calls to GitHub:

```
import { Readable } from 'node:stream';

async function* ghRepos() {
  const response = await fetch('https://api.github.com/users');
  const users = await response.json();

  for (let index = 0; index < 10; index++) {
    const reposResponse = await fetch(users[index].repos_url);
    yield await reposResponse.json();
  }
}
```

```
async function* ghFirsts() {
  for await (const repos of ghRepos()) {
    if (repos[0]) {
      yield repos[0].full_name + '\n';
    }
  }
}

Readable.from(ghFirsts()).pipe(process.stdout);
```

TIP

To better understand this example, look at <https://api.github.com/users> in your browser, find a user's `repos_url`, and look at that URL too.

The `ghRepos` function first fetches a list of users, then iteratively fetches (and then yields) the list of repos for the first 10 users. The `ghFirsts` function is then used to yield the name of the first repo from the list yielded by the `ghRepos` function.

Since the `ghFirsts` function is an async generator, we can use it to create a readable stream where each chunk is the name of the first repo for each user.

The `pipeline` method even supports having an async generator function in the mix. Instead of the last line, where we used `.pipe`, you can directly include `ghFirsts` within a `pipeline` call:

```
// Instead of:
// Readable.from(ghFirsts()).pipe(process.stdout);

await pipeline(ghFirsts(), process.stdout);
```

[Figure 6-6](#) shows the output.

```
JS async-generator.js X
JS async-generator.js > ...
1  import { pipeline } from 'node:stream/promises';
2
3  > async function* ghRepos() { ...
11 }
12
13 > async function* ghFirsts() { ...
19 }
20
21 await pipeline(ghFirsts(), process.stdout);
22

TERMINAL
bash + v [ ] [ ] ...
● $ node async-generator.js
mojombo/30daysoflaptops.github.io
defunkt/ace
pjhyett/auto_migrations
wycats/abbot-from-scratch
ezmobius/acl_system2
ivey/advent-of-code-2022
evanphx/ace
vanpelt/amqp
wayneeseguine/Adv360-Pro-ZMK
brynary/active_admin
○ $
```

Figure 6-6. An async generator readable stream

The `pipeline` method also passes a stream object to an async generator that follows it in the pipeline, effectively enabling an async generator to act like a transform stream. Here's the uppercase transform example done with an async generator function:

```
import { pipeline } from 'node:stream/promises';

async function* upperCaseTr(source) {
  for await (const chunk of source) {
    yield String(chunk).toUpperCase();
  }
}

await pipeline(
  process.stdin,
  upperCaseTr,
  process.stdout
);
```

Readable streams can be consumed using an async iterator too:

```
for await (const chunk of readableStream) {  
  console.log(chunk);  
}
```

Streams Object Mode

By default, streams expect buffer or string values. There is an `objectMode` flag that we can set to have the stream accept any JavaScript object.

Here's a simple example to demonstrate that. The following combination of transform streams makes a feature to map a string of comma-separated values into a JavaScript object. So `a,b,c,d` becomes `{a: b, c: d}`:

```
import { Transform } from 'node:stream';  
  
const commaSplitter = new Transform({  
  readableObjectMode: true,  
  
  transform(chunk, encoding, callback) {  
    this.push(chunk.toString().trim().split(','));  
    callback();  
  },  
});  
  
const arrayToObject = new Transform({  
  readableObjectMode: true,  
  writableObjectMode: true,  
  transform(chunk, encoding, callback) {  
    const obj = {};  
    for (let i = 0; i < chunk.length; i += 2) {  
      obj[chunk[i]] = chunk[i + 1];  
    }  
    this.push(obj);  
    callback();  
  },  
});
```



```
const objectToString = new Transform({
  writableObjectMode: true,
  transform(chunk, encoding, callback) {
    this.push(JSON.stringify(chunk) + '\n');
    callback();
  },
});
```

You can use these transform streams with the `pipeline` method:

```
await pipeline(
  process.stdin,
  commaSplitter,
  arrayToObject,
  objectToString,
  process.stdout,
);
```

We pass an input string (for example, `a,b,c,d`) through `commaSplitter`, which pushes an array as its readable data (`["a", "b", "c", "d"]`). Adding the `readableObjectMode` flag on that stream is necessary because we're pushing an object there, not a string.

We then take the array and pipe it into the `arrayToObject` stream. We need a `writableObjectMode` flag to make that stream accept an object. It'll also push an object (the input array mapped into an object), and that's why we also need the `readableObjectMode` flag there as well. The last `objectToString` stream accepts an object but pushes out a string, and that's why we need only a `writableObjectMode` flag there. The readable part is a normal string (the `stringified` object; see [Figure 6-7](#)).

```
JS stream-object-mode.js X
JS stream-object-mode.js > ...
1  import { Transform } from 'node:stream';
2  import { pipeline } from 'node:stream/promises';
3
4  > const commaSplitter = new Transform({...
11  });
12
13  > const arrayToObject = new Transform({...
24  });
25
26  > const objectToString = new Transform({...
32  });
33
34  await pipeline(
35    process.stdin,
36    commaSplitter,
37    arrayToObject,
38    objectToString,
39    process.stdout,
40  );
41

TERMINAL
○ $ node stream-object-mode.js
a,b,c,d
{"a":"b","c":"d"}
```

Figure 6-7. Stream object mode

Built-In Transform Streams

Node has a few very useful built-in transform streams such as the `zlib` and `crypto` streams.

Here's an example that uses the `zlib.createGzip()` stream combined with the `node:fs` readable/writable streams to create a file-compression script:

```
import fs from 'node:fs';
import zlib from 'node:zlib';
import { pipeline } from 'node:stream/promises';

const file = process.argv[2];

await pipeline(
  fs.createReadStream(file),
  zlib.createGzip(),
```

```
fs.createWriteStream(file + '.gz')
);
```

You can use this script to gzip any file you pass its path as the argument. We're piping a readable stream for that file into the zlib built-in transform stream and then into a writable stream for the new gzipped file.

The cool thing about a pipeline of streams is that it enables us to compose our program piece by piece, in a readable way. Say, for example, you want the user to see a progress indicator while the file is being compressed. You can create a stream to report progress and make it part of the pipeline. An async generator function makes tasks like these easy:

```
await pipeline(
  fs.createReadStream(file),
  zlib.createGzip(),
  async function* (source) {
    for await (const chunk of source) {
      process.stdout.write('.');
      yield chunk;
    }
  },
  fs.createWriteStream(file + '.gz')
);
```

The applications of combining streams are endless. For example, if we need to encrypt the file before (or after) we gzip it, all we need to do is add another transform stream to the pipeline. The `node:crypto` module has the `createCipheriv` method that can be used for that purpose:

```
import crypto from 'node:crypto';

// ..

const algorithm = 'aes-256-ctr';
const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(16);

await pipeline(
```

```
fs.createReadStream(file),
zlib.createGzip(),
crypto.createCipheriv(algorithm, key, iv),
fs.createWriteStream(file + '.gz'),
);
```

This code compresses and then encrypts the file it receives as an argument, which means the file can't even be unzipped with a normal unzip utility. Only someone who has the `key/iv` values can use this file. To actually be able to unzip anything zipped with this code, we need to use the opposite streams for `crypto` and `zlib` in a reverse order:

```
await pipeline(
  fs.createReadStream(file),
  crypto.createDecipheriv(algorithm, key, iv),
  zlib.createGunzip(),
  fs.createWriteStream(file.slice(0, -3)),
);
```

This code creates a read stream from the encrypted and zipped file, pipes it into the `crypto.createDecipheriv()` stream (using the same `key/iv` values), pipes the output of that into the `zlib.createGunzip()` stream, and then writes the content out back to a file without the `.gz` extension part.

Summary

Streams provide a memory-efficient way to process large data in Node. They allow us to work with data in chunks over time instead of needing to handle all the data at once.

The two main types of streams are readable streams (which produce data) and writable streams (which consume data). Duplex streams combine both functionalities, and transform streams are duplex streams that modify data in transit as well.

Streams have many methods and events that can be used together to ensure proper use. Readable streams can be in a flowing mode or a paused mode. A writable stream can be faster or slower than a readable stream. All these different states require the use of different methods and events. The easiest way to work with streams is through the use of the `pipeline` method as it automatically handles many of the complexities around using streams.

In the next chapter, we'll explore how to make a Node process execute a command from the OS and how to make a process fork another one.